PLAN DE TRABAJO · CIVIL3D-MCP · CIVIL 3D 2027 · VERSIÓN PARA GITHUB

Hoja de Ruta con Prompts v1.3

Claude Code web en el navegador + agente instalador con el MCP

Octubre 2026 · Reemplaza a la v1.1 y la v1.2 · Las pruebas en Civil 3D las hace tu agente con el MCP
Incluye al final los ajustes a la hoja de ruta general «IA especialista en ingeniería» v1.1

QUÉ CAMBIÓ EN LA VERSIÓN 1.3

- **Claude Code web trabaja en la nube.** No ve tu PC, ni Civil 3D, ni las DLL de Autodesk, ni tu MCP. Escribe el código, corre las pruebas que no necesitan Civil 3D y sube una rama a GitHub.
- **Tu agente instalador hace lo que necesita tu PC:** encontrar el MCP viejo, compilar con las DLL reales, instalar, correr el extractor y subir la memoria.
- **Tú usas Claude solo en la web.** Las pruebas en Civil 3D las hace tu agente instalador, usando **solo** el MCP civil3d (Prompt P). Si usa curl o scripts, la prueba no vale.
- **Siete prompts nuevos:** L.0 (encontrar el MCP viejo y preparar la PC, ya hecho), 0.5 (reglas de GitHub), 0.6 (memoria-firma lista), L.1 (instalar una rama), P (probar en Civil 3D con el MCP), L.2 (sincronizar la memoria) y F (fichas desde la nube).
- **buscar_api también funciona como comando,** para que Claude Code web consulte el catálogo en la nube antes de programar.
- **El código solo se arregla en la nube.** El agente instalador instala, corre y reporta. Así hay una sola versión: la de GitHub.

1 Quién hace qué

Piensa en un carpintero y un instalador. El carpintero (Claude Code web) fabrica la puerta en su taller. El instalador (tu agente) la lleva a tu casa, la monta y la abre y cierra para probarla. Tú miras que quede bien. Si funciona, el plano se archiva (Merge en GitHub).

Quién	Dónde está	Qué hace	Qué NO hace
Claude Code web	La nube (claude.ai/code)	Escribe código, corre pruebas, crea ramas y Pull Requests	No toca Civil 3D ni tu disco
Agente instalador	Tu PC	Descarga ramas, compila con las DLL reales, instala, prueba en Civil 3D usando solo el MCP civil3d, corre el evaluador y sube la memoria	No cambia código ni habla con Civil 3D por fuera del MCP
Tú	Claude en la web y tu PC	Apruebas planes, miras el resultado en Civil 3D y haces Merge	

2 Antes de empezar (una sola vez)

☐ Instala la **Claude GitHub App** en tu repositorio privado memoria-firma (github.com/apps/claude). Sin ella, Claude Code web no puede verlo.
☐ Al abrir cada sesión en claude.ai/code, usa el **selector de repositorio** debajo del cuadro de texto. Una sesión puede tener varios repositorios: agrega los que diga la línea *Repos* de cada prompt.
☐ En el selector de permisos elige **Plan**: Claude Code propone el plan y espera tu aprobación antes de tocar archivos.
☐ Después del prompt 0.5, abre el selector de entorno (debajo del cuadro de texto), entra a los ajustes de tu entorno y pega en **Setup script** el texto que te dé Claude Code. Así cada sesión arranca con .NET 10 instalado (la nube no lo trae).

3 El ciclo de cada prompt

#	Quién	Qué haces
1	Tú	Abres una sesión nueva en claude.ai/code con los repos que indica el prompt y el modo Plan.
2	Claude Code web	Pegas el prompt. Lees el plan y respondes «aprobado» (o lo corriges).
3	Claude Code web	Trabaja en una rama tarea/..., corre las pruebas y la sube. Si no abrió el PR, pulsa **Create PR** (borrador) en la vista de cambios.
4	Agente instalador	Pegas el Prompt L.1 con el nombre de la rama.
5	Agente instalador	Pegas el Prompt P. El agente prueba con el MCP siguiendo «Para Jose» del PR y tú miras el resultado en Civil 3D.
6	Claude Code web	Si falla, pegas en la **misma** sesión: «Falló esto: [informe]. Corrígelo en la misma rama sin cambiar nada más». Vuelves al paso 4.
7	Tú	Si pasa, escribes «Verificación OK» en la sesión, pulsas **Merge pull request** en GitHub y corres L.1 con main.

Los prompts que no tocan Civil 3D (0.5, 0.6, 1.2, 3.3, 4.1 y F) se saltan los pasos 4 y 5.

4 Orden de ejecución actualizado

Paso	Prompts	Hito para avanzar
Puesta al día	L.0 · 0.5 · 0.6	MCP nuevo funcionando en Civil 3D, el viejo en respaldo, CLAUDE.md con las reglas de GitHub y memoria-firma visible en la nube
Etapa 1	1.1 · 1.2 · 1.3	buscar_api responde por el MCP y como comando en la nube
Registro	5.1	La primera tarea aprobada aparece en pares.jsonl
Línea base	5.2	Primer informe del evaluador con las 10 tareas (la mayoría en FALLO es normal)
Etapa 2	2.1 a 2.8	La tarea 10 (flujo completo) da OK
Etapa 3	3.1 · 3.2 · 3.3 (mensual)	5 recetas aprobadas y reutilizadas
Etapa 4	4.1 · 4.2 · 4.3	Verifica un eje real y cita la tabla de la norma
Continuo	F · L.2 · 5.2	Una ficha por proyecto cerrado; L.2 después de guardar algo en tu PC; evaluador tras cada versión

HAZLOS EN ESTE ORDEN, ANTES DE LA ETAPA 1

5 Puesta al día: prompts nuevos

Prompt L.0 · Encontrar el MCP viejo y preparar tu PC NUEVO

DÓNDE: AGENTE INSTALADOR (TU PC)

Ya lo hiciste. Queda aquí como referencia.

```
Trabajas en mi PC con Windows. Yo no programo: explícame todo en lenguaje sencillo.
Regla principal: NO borres nada. Antes de mover, desactivar o cambiar cualquier cosa, muéstrame
lo que encontraste y espera mi "sí".

Parte A · Encontrar la versión vieja de mi MCP de Civil 3D (solo buscar, no tocar)
1. Busca en todos mis discos carpetas o archivos llamados civil3d-mcp, ArbaMcp, PuenteMcp o
   ArbaMcp*.dll, y carpetas .bundle de ArbaMcp en %APPDATA%\Autodesk\ApplicationPlugins,
   C:\ProgramData\Autodesk\ApplicationPlugins y C:\Program Files\Autodesk\ApplicationPlugins.
2. Revisa la configuración de Claude Desktop (%APPDATA%\Claude\claude_desktop_config.json) y, si
   existen, %USERPROFILE%\.claude.json y archivos .mcp.json. Dime qué servidor MCP apunta a
   PuenteMcp y con qué ruta. No muestres tokens ni claves en tu respuesta.
3. Revisa si Civil 3D carga el plugin al iniciar (claves Applications de AutoCAD en el registro,
   acad.lsp, acaddoc.lsp o el grupo de inicio). Solo léelo.
4. Por cada copia encontrada dime: ruta, fecha, versión del DLL y, si es un repositorio git:
   rama, último commit, dirección del remoto y si tiene cambios sin subir (git status y
   git log origin/main..HEAD). Si tiene cambios que no están en GitHub, avísame en grande:
```

```
   no se pueden perder.
5. Si encuentras más de una copia, no adivines cuál es la buena: muéstrame una tabla.

Parte B · Comprobar programas
6. Dime qué versión tengo de git, .NET SDK (necesito .NET 10), Python, pip, Civil 3D 2027 (su
   ruta de instalación, la que el proyecto llama C3DPath). Dime también si
   para compilar basta el .NET 10 SDK o hace falta Visual Studio. Si falta algo, dime qué es y
   cómo lo instalarías, y espera mi aprobación.

Parte C · Preparar la versión nueva (solo después de mi "sí")
7. Clona https://github.com/[TU_USUARIO_GITHUB]/civil3d-mcp en C:\Proyectos\civil3d-mcp. Si esa
   carpeta ya existe, no la pises: dime qué hay.
8. Clona mi repositorio privado memoria-firma en C:\Proyectos\memoria-firma. Si ya existe una
   copia local, comprueba que su remoto es el de GitHub y que está al día; si hay diferencias,
   muéstramelas y no sobrescribas nada.
9. Mueve la versión vieja a C:\Proyectos\_respaldo_mcp_viejo_[FECHA] (no la borres) y quita su
   carga automática, para que no compita con la nueva (mismos comandos y mismo puerto).
10. Sigue ArbaMcp/LEEME.md para compilar con las DLL reales de Civil 3D 2027, correr las
    pruebas (dotnet test y pytest), instalar el plugin y conectar el MCP civil3d en tu propia
    configuración de MCP. Guarda una copia antes de editarla.
11. Crea C:\Proyectos\ESTADO_PC.md con todo lo encontrado y hecho: rutas, versiones, rama
    instalada y resultado de las pruebas. No escribas ahí tokens ni contraseñas.

Al final dame una tabla de lo hecho y los pasos exactos para comprobar que en Civil 3D aparece
el botón Conexión IA y que tú ves las herramientas del MCP civil3d.
```

Verificación: existe C:\Proyectos\ESTADO_PC.md; la versión vieja está en la carpeta _respaldo; en Civil 3D aparece Conexión IA y el agente, usando solo el MCP civil3d, lista las superficies del dibujo.

Prompt 0.5 · Reglas para trabajar en GitHub NUEVO

DÓNDE: CLAUDE CODE WEB · Repos: civil3d-mcp

```
No cambies código del plugin ni del puente en esta sesión.
Contexto nuevo: ahora trabajo contigo en Claude Code web. Tú trabajas en la nube sobre una copia
de este repositorio: no tienes Civil 3D, ni las DLL de Autodesk, ni acceso a mi PC, ni al MCP.
En mi PC hay un "agente instalador" que descarga tus ramas, compila con las DLL reales e instala
el plugin, y luego ese mismo agente prueba las herramientas en Civil 3D usando solo el MCP
civil3d.

1. Actualiza CLAUDE.md. Conserva las 10 reglas y agrega la sección "Trabajo en GitHub":
   a. Nunca trabajes directo en main: una rama por tarea (tarea/[numero]-[nombre-corto]),
      commits pequeños con mensajes en español, push y Pull Request en borrador. Si no puedes
      abrir el PR, dímelo y lo abro yo con el botón Create PR.
   b. Corre en la nube lo que no necesita Civil 3D: pruebas de ArbaMcp.Nucleo, pytest de
      PuenteMcp y CompilarSinCivil. Si algo no se puede correr aquí, dilo; no lo des por bueno.
   c. Nunca digas que algo funciona en Civil 3D: eso solo lo confirmo yo.
   d. No inventes miembros de la API de Civil 3D. Confírmalos en el catálogo de memoria-firma
      (cuando esté en la sesión) y anota en miembros_por_verificar_civil3d.md lo que no puedas
      confirmar.
   e. Cada PR lleva dos secciones: "Para el agente instalador" (rama, qué compilar, qué
      instalar, qué variables de entorno poner, qué reiniciar) y "Para Jose" (qué abrir en
      Civil 3D, qué pedirle al agente que pruebe con el MCP, qué debería salir y qué archivo
      VALIDACION usar).
   f. Cuando yo te pegue el informe del agente instalador con errores, corrige en la misma rama.
   g. El PR pasa a "listo" solo cuando yo diga que la verificación en Civil 3D pasó. El Merge lo
      hago yo en GitHub.
   h. Commits, nombres de rama y descripciones de PR también son públicos: nada de datos de
      clientes, rutas de mi PC con nombres de proyectos ni contenido de memoria-firma.
   i. memoria-firma es un repositorio privado aparte. Si la tarea lo necesita y no lo ves en
      esta sesión, dímelo antes de empezar.
2. Crea scripts/preparar_nube.sh que instale el .NET 10 SDK y las dependencias de Python de
   PuenteMcp. Usa apt (los dominios de Microsoft y de Ubuntu están permitidos en la red de la
   nube). Debe tardar menos de 5 minutos, no depender de archivos del repo y terminar siempre
   con exit 0, mostrando un aviso claro si algo falló. Pruébalo aquí y dime qué pasó. Al final
   dame su contenido completo para pegarlo en el campo "Setup script" de mi entorno.
3. Revisa que el CI de GitHub Actions corra las pruebas en cada Pull Request. Si no lo hace,
   proponme el cambio y espera mi aprobación.
4. Crea herramientas-dev/PLANTILLA_PR.md con las secciones del punto 1e.

Trabaja en la rama tarea/0.5-github, abre el PR y explícame en lenguaje sencillo cómo será
nuestro ciclo de trabajo desde ahora.
```

Verificación: en GitHub ves el PR de tarea/0.5-github, CLAUDE.md tiene la sección «Trabajo en GitHub» y pegaste el Setup script en tu entorno. Abre una sesión nueva y pide «corre dotnet --version»: debe decir 10. Haz Merge.

Prompt 0.6 · memoria-firma lista para la nube NUEVO

DÓNDE: CLAUDE CODE WEB · Repos: memoria-firma

```
Esta sesión es sobre mi repositorio PRIVADO memoria-firma. No toques civil3d-mcp.
Yo no programo: explícame en lenguaje sencillo y muéstrame el plan antes de cambiar nada.
1. Revisa que exista lo de la Etapa 0: catalogo/, recetas/, normas/fuentes/, normas/tablas/,
   casos/, evaluacion/, evaluacion/informes/, dataset/, README.md, .gitignore,
   evaluacion/tareas.yaml con mis 10 tareas, casos/PLANTILLA_FICHA.md, COMO_CAPTURAR.md,
   validar_fichas.py, PROHIBIDOS.txt y mi primera ficha. Dime qué falta; no lo rehagas sin
   preguntarme.
2. Crea CLAUDE.md para este repositorio: es privado y guarda mi memoria técnica; yo no
   programo; plan antes de cambiar; una tarea por sesión; toca solo lo necesario; nunca
   agregues .dwg, planos ni datos sin anonimizar; corre validar_fichas.py antes de cada commit
   que toque casos/. Aquí se puede trabajar directo en main.
3. Agrega una GitHub Action que corra validar_fichas.py en cada push.
4. Crea sincronizar_memoria.ps1 (lo correrá mi agente instalador en Windows): git pull; corre
   validar_fichas.py; si falla, se detiene y dice por qué; si pasa, agrega solo catalogo/,
   recetas/, normas/, casos/, evaluacion/ y dataset/ (nunca .dwg ni archivos de más de 50 MB),
   hace commit con la fecha y push normal (nunca --force). Al final muestra qué subió.
5. Agrega al .gitignore lo que falte para bloquear .dwg, .dwl, .dwl2, .bak y .sv$.
6. Si ya hay subido algún .dwg o archivo de más de 50 MB, avísame.
```

Verificación: en GitHub, memoria-firma (Private) tiene CLAUDE.md, sincronizar_memoria.ps1 y la Action en verde.

Prompt L.1 · Instalar y probar una rama REUTILIZABLE

DÓNDE: AGENTE INSTALADOR (TU PC)

```
Lee C:\Proyectos\ESTADO_PC.md para recordar cómo está mi PC. Luego:
1. Si Civil 3D está abierto, pregúntame antes de cerrarlo (puede haber dibujos sin guardar).
2. En C:\Proyectos\civil3d-mcp: git fetch y cambia a la rama [NOMBRE_DE_LA_RAMA] (o main).
   Si hay cambios locales sin subir, detente y avísame.
3. Lee la sección "Para el agente instalador" del Pull Request de esa rama y síguela.
4. Compila con las DLL reales de Civil 3D 2027, corre dotnet test y pytest, e instala el plugin
   y el puente como dice ArbaMcp/LEEME.md.
5. Si cambió algo del puente o de las variables de entorno, reinicia el puente
   (startup_c3d_mcp.vbs) y vuelve a conectar tu MCP civil3d.
   Todavía no pruebes nada en Civil 3D: eso es el Prompt P.
6. Actualiza ESTADO_PC.md con la rama y la versión instaladas.
No intentes arreglar el código si algo falla: solo informa.
Entrégame un informe corto que pueda pegar tal cual en Claude Code web: rama, commit, resultado
de la compilación, pruebas que pasaron y fallaron, y los errores completos si hubo.
```

Verificación: el informe dice la rama y el commit correctos y no hay errores de compilación.

Prompt L.2 · Sincronizar memoria-firma REUTILIZABLE

DÓNDE: AGENTE INSTALADOR (TU PC)

Córrelo cada vez que guardes algo en tu PC: fichas, recetas, informes, dataset o catálogo.

```
En C:\Proyectos\memoria-firma corre sincronizar_memoria.ps1. Si validar_fichas.py falla,
muéstrame qué ficha y por qué, y no subas nada. Al final dime qué archivos subió a GitHub.
```

Verificación: en GitHub, memoria-firma muestra un commit nuevo con la fecha de hoy.

Prompt P · Probar en Civil 3D con el MCP REUTILIZABLE

DÓNDE: AGENTE INSTALADOR (TU PC)

Va siempre después de L.1. Ten abierto Civil 3D con una copia de tu dibujo de prueba.

```
Prueba en Civil 3D la rama [NOMBRE_DE_LA_RAMA] que acabas de instalar.
Regla de oro: usa SOLO las herramientas del MCP civil3d. No llames a Civil 3D con curl,
scripts ni el puerto 8765. Si una herramienta no aparece o falla, detente y dímelo: eso
también es un resultado.
1. Comprueba que Civil 3D está abierto con una COPIA de mi dibujo de prueba y que pulsé
   Conexión IA. Si no, pídemelo y espera.
2. Lista las herramientas que ves en el MCP civil3d y confirma que están las nuevas de esta
   rama.
3. Sigue la sección "Para Jose" del Pull Request y, si existe,
   herramientas-dev/VALIDACION_[NOMBRE].md, paso a paso. Primero con simular=true.
4. Antes de cada paso que cambie el dibujo, avísame para que yo mire Civil 3D.
5. No arregles código.
Entrégame un informe corto para pegar en Claude Code web: herramientas usadas, OK/FALLO por
paso, errores completos y tiempos.
```

Verificación: el informe dice qué herramientas del MCP usó, con OK o FALLO en cada paso, y no menciona curl ni el puerto 8765.

SEMANAS 2 Y 3

6 Etapa 1 · Catálogo, glosario y buscar_api

Prompt 1.1 · Extractor del catálogo AJUSTADO

DÓNDE: CLAUDE CODE WEB · Repos: civil3d-mcp

```
Revisa herramientas-dev/InspectC3D y ArbaMcp.Nucleo/Api. Con esa base, crea una herramienta de
consola en herramientas-dev/ExtractorCatalogo que:
1. Lea las DLL de AutoCAD y Civil 3D 2027 SOLO como metadatos, sin cargarlas ni ejecutarlas
   (System.Reflection.MetadataLoadContext; si no alcanza, Mono.Cecil). La carpeta de las DLL y
   el archivo de salida llegan como parámetros (--dll y --salida). No escribas en el código
   ninguna ruta de mi PC.
2. Guarde en SQLite (catalogo_c3d2027.sqlite): espacios de nombres, clases, métodos,
   propiedades, parámetros con tipos, valor de retorno, si es estático, descripción y fuente
   de la descripción. Si no hay XML de documentación, deja la descripción vacía.
3. Cree un índice FTS5 sobre nombres y descripciones y deje creada la tabla glosario
   (termino, sinonimos, concepto_api, nota) vacía.
4. Al terminar imprima cuántas clases, métodos y propiedades extrajo y el tamaño del archivo.
Aquí en la nube no hay DLL de Autodesk y nunca deben llegar al repositorio. Para las pruebas,
crea en el proyecto de pruebas una biblioteca pequeña de ejemplo (dos o tres clases falsas con
métodos, propiedades y un método estático de creación) y extrae esa. Agrega al .gitignore
cualquier .sqlite dentro de herramientas-dev.
En "Para el agente instalador" del PR escribe el comando exacto para correr el extractor en mi
PC con las DLL reales y guardar el resultado en C:\Proyectos\memoria-firma\catalogo\, y 3
consultas SQL de ejemplo para comprobarlo (una por TinSurface).
```

Prompt 1.1-PC · Correr el extractor en tu PC NUEVO

DÓNDE: AGENTE INSTALADOR (TU PC)

```
(Pega L.1 con el nombre de la rama y agrega al final:)
Además: corre el extractor como dice "Para el agente instalador" del PR, corre las 3 consultas
de ejemplo y muéstrame los resultados. Si catalogo_c3d2027.sqlite pesa menos de 50 MB, corre
sincronizar_memoria.ps1 para subirlo a memoria-firma. Si pesa más, avísame y no lo subas.
```

Verificación: la consulta por TinSurface devuelve la clase y su método de creación, y en GitHub aparece memoria-firma/catalogo/catalogo_c3d2027.sqlite. Escribe «Verificación OK» en la sesión web y haz Merge.

Prompt 1.2 · Glosario español → API AJUSTADO

DÓNDE: CLAUDE CODE WEB · Repos: memoria-firma + civil3d-mcp

```
Esta sesión tiene dos repositorios: memoria-firma (aquí trabajas, directo en main) y civil3d-mcp
(solo para leer). Toma la semilla del glosario del punto 4 de
civil3d-mcp/herramientas-dev/AUDITORIA_FLUJO_VIAL.md y esta lista mía:
[pega tus términos; por ejemplo: rasante, eje, progresiva, PI, PC, PT, peralte, sobreancho,
talud, cota, curva de nivel, sección transversal, ensamblaje, subensamblaje, línea de muestreo,
corte, relleno, esponjamiento, curva vertical, PIV, visibilidad de parada]
1. Crea catalogo/glosario.yaml. Por cada término: termino, sinonimos (incluye el término en
   inglés), concepto_api (clase, método o propiedad) y una nota de una línea. Mínimo 80.
2. Comprueba cada concepto_api contra catalogo/catalogo_c3d2027.sqlite. Si no existe en el
   catálogo, márcalo «por confirmar». Nunca inventes.
3. Crea catalogo/cargar_glosario.py, que borre y vuelva a cargar la tabla glosario desde el YAML
   (para poder correrlo muchas veces), con pruebas pytest sobre un catálogo pequeño de ejemplo.
4. Córrelo sobre el catálogo real aquí mismo y dime cuántos términos quedaron confirmados y
   cuántos «por confirmar». Luego haz commit y push.
```

Verificación: en la misma sesión pide «consulta rasante, progresiva y talud en la tabla glosario». Rasante debe dar Profile (de diseño), progresiva debe dar station y talud debe dar daylight / objetivo de superficie.

Prompt 1.3 · Herramienta buscar_api AJUSTADO

DÓNDE: CLAUDE CODE WEB · Repos: civil3d-mcp + memoria-firma

```
Esta sesión tiene dos repositorios: civil3d-mcp (aquí programas, en una rama) y memoria-firma
(solo para leer y probar). Agrega a PuenteMcp la herramienta buscar_api(consulta, limite=10):
1. Expande la consulta con la tabla glosario (español → términos de la API) antes de buscar.
2. Busca en el catálogo y en recetas/ de memoria-firma. La carpeta de memoria-firma llega por la
   variable de entorno MEMORIA_FIRMA_DIR (catálogo en catalogo/, recetas en recetas/). Si la
   variable no está o falta el archivo, responde un error claro, sin romperse.
3. Devuelve resultados compactos: clase, firma, parámetros y una línea de descripción. Orden:
   recetas, luego glosario, luego el resto del catálogo.
4. Funciona aunque Civil 3D esté cerrado.
5. Se puede usar también como comando, sin MCP (python -m [módulo] buscar_api "consulta"),
   para que tú mismo la uses en la nube en las próximas tareas.
Pruebas pytest con un catálogo y un glosario pequeños de ejemplo (nunca el catálogo real dentro
de civil3d-mcp). Como memoria-firma está en esta sesión, prueba también el comando con mi
catálogo real: "rasante con PVI", y muéstrame el resultado.
Actualiza INSTRUCCIONES_AGENTE.md con el orden: herramienta existente > receta > buscar_api +
código > ejecutar_comando como último recurso. Agrega a CLAUDE.md: "Antes de usar un miembro de
la API, búscalo con el comando de buscar_api".
En "Para el agente instalador": cómo poner MEMORIA_FIRMA_DIR=C:\Proyectos\memoria-firma como
variable de entorno de mi usuario en Windows y reiniciar el puente (el puente arranca solo al
encender la PC; no lo abre el cliente MCP).
```

Verificación: corre L.1 y luego el Prompt P pidiendo con buscar_api «cómo crear una rasante con PVI»: debe responder con pocas firmas relevantes (perfil por layout, PVI, curvas verticales), sin explorar la API.

VA JUSTO DESPUÉS DE LA ETAPA 1

7 Etapa 5 · Registro y evaluador

Prompt 5.1 · Registro que sirve como dataset AJUSTADO

DÓNDE: CLAUDE CODE WEB · Repos: civil3d-mcp

```
Extiende el registro para que sirva como dataset:
1. Todas las herramientas aceptan un campo opcional pedido (mi pedido original en lenguaje
   natural). Actualiza INSTRUCCIONES_AGENTE.md para que el agente lo envíe siempre.
2. Herramienta aprobar_resultado(id_log, aprobado, comentario) para que yo marque si un
   resultado fue correcto.
3. Script exportar_dataset que lea los mcp_log.jsonl, tome solo lo aprobado, quite rutas y
   nombres de proyecto, y escriba pares.jsonl con: pedido, herramientas o receta usadas,
   parámetros y resultado. La carpeta de logs y la de salida llegan como parámetros; por
   defecto la salida es MEMORIA_FIRMA_DIR/dataset/pares.jsonl.
Pruebas aquí en la nube con logs de ejemplo que incluyan rutas y nombres de proyecto falsos,
para comprobar que se borran. En "Para el agente instalador": el comando exacto para correr
exportar_dataset en mi PC y, después, sincronizar_memoria.ps1.
```

Verificación: corre L.1. Con el Prompt P, pide al agente 2 o 3 tareas con el MCP y apruébalas con aprobar_resultado. Pide al agente instalador que corra exportar_dataset y L.2. En GitHub,

memoria-firma/dataset/pares.jsonl tiene una línea por tarea y ninguna ruta ni nombre de proyecto.

Prompt 5.2 · Evaluador automático (línea base) AJUSTADO

DÓNDE: CLAUDE CODE WEB · Repos: civil3d-mcp + memoria-firma

```
Esta sesión tiene dos repositorios: civil3d-mcp (aquí programas, en una rama) y memoria-firma.
Crea herramientas-dev/Evaluador (Python) con dos comandos:
1. generar: lee MEMORIA_FIRMA_DIR/evaluacion/tareas.yaml y escribe un prompt de evaluación para
   mi agente instalador, que usa el MCP civil3d: usar SOLO herramientas del MCP, ejecutar cada
      tarea sobre una COPIA de mi dibujo
   de prueba, primero con simular=true, comparar con resultado_esperado y terminar con un
   informe OK/FALLO con tiempos en un bloque YAML de formato fijo (para poder guardarlo).
2. comparar: recibe ese bloque YAML, lo guarda con fecha en evaluacion/informes/ y lo compara
   con el informe anterior: cuántas tareas pasaron de FALLO a OK (o al revés) y cómo cambiaron
   los tiempos.
Pruebas pytest con un tareas.yaml y dos informes de ejemplo. Luego corre generar con mis 10
tareas reales y guarda el resultado en memoria-firma/evaluacion/PROMPT_EVALUACION.md (commit y
push en memoria-firma). En "Para el agente instalador": cómo correr comparar en mi PC.
```

Prompt 5.2-D · Correr la evaluación NUEVO

DÓNDE: AGENTE INSTALADOR (TU PC)

Lo hace tu agente instalador con Civil 3D abierto, usando solo el MCP. Al final te da un bloque YAML.

```
(Abre en Civil 3D una COPIA de tu dibujo de prueba. En GitHub abre
memoria-firma/evaluacion/PROMPT_EVALUACION.md, copia todo y pégalo en tu agente instalador.
Agrega al final: Usa solo las herramientas del MCP civil3d, nunca curl ni scripts.)
```

Prompt 5.2-PC · Guardar el informe NUEVO

DÓNDE: AGENTE INSTALADOR (TU PC)

```
Guarda este informe con el comando comparar del Evaluador
(C:\Proyectos\civil3d-mcp\herramientas-dev\Evaluador) y muéstrame la comparación con el
anterior. Luego corre sincronizar_memoria.ps1.
[pega aquí el bloque YAML del informe de la evaluación]
```

Verificación: el primer informe aparece en memoria-firma/evaluacion/informes/ con tus 10 tareas. Repite 5.2-D y 5.2-PC después de cada versión nueva del plugin.

SEMANAS 4 A 15 · UNA HERRAMIENTA POR SESIÓN

8 Etapa 2 · Herramientas de creación del flujo vial

Usas siempre la misma plantilla y solo cambias el nombre y la especificación. Las especificaciones no cambian respecto a la v1.1; están aquí para que tengas todo en un solo documento.

Prompt 2.x · Plantilla para cada herramienta AJUSTADO

DÓNDE: CLAUDE CODE WEB · Repos: civil3d-mcp + memoria-firma

```
Esta sesión tiene dos repositorios: civil3d-mcp (aquí programas) y memoria-firma (solo para
consultar el catálogo). Implementa la herramienta [NOMBRE] en el plugin, en la rama
tarea/2.[N]-[nombre], siguiendo el patrón de las herramientas de escritura existentes
(Escritura.cs: comprobaciones, copia, simular, antes/después, Deshacer y log) y las reglas de
CLAUDE.md.
Especificación:
[PEGA AQUÍ LA ESPECIFICACIÓN]
Antes de programar, usa el comando de buscar_api para confirmar las clases y métodos de Civil 3D
2027 que necesitas y anótalos en miembros_por_verificar_civil3d.md como «por verificar». Si un
miembro no aparece en el catálogo, no lo uses sin decírmelo.
Pon en ArbaMcp.Nucleo la lógica que no depende de Civil 3D (validar parámetros, cálculos, armar
la respuesta), con pruebas que corran aquí. Corre dotnet test, pytest y CompilarSinCivil antes
del push.
Al terminar: actualiza CONTRATO.md, LEEME.md e INSTRUCCIONES_AGENTE.md, sube la versión, crea
herramientas-dev/VALIDACION_[NOMBRE].md con un prompt de validación (informe OK/FALLO) sobre mi
dibujo de prueba, con un pedido de ejemplo en español tal como yo lo diría, y abre el PR con
"Para el agente instalador" y "Para Jose".
Recuerda: aquí no puedes compilar contra Civil 3D. Los errores reales te llegarán en el informe
del agente instalador; cuando te lo pegue, corrige en la misma rama.
```

Especificaciones (pega una en la plantilla)

2.1 · crear_superficie_tin SIN CAMBIOS

```
Crea una superficie TIN a partir de un grupo de puntos (por nombre) o de un archivo de puntos.
Parámetros: nombre, grupo_puntos o archivo, estilo, intervalo de curvas menor y mayor. Devuelve
número de puntos, de triángulos y extensión. Si ya existe una superficie con ese nombre,
responde error sin tocar nada.
```

2.2 · crear_alineamiento_desde_polilinea SIN CAMBIOS

```
Crea un alineamiento desde una polilínea (por handle o capa). Parámetros: nombre, estilo,
conjunto de etiquetas, radio por defecto y, opcionalmente, una lista de radios por PI. Agrega
una curva en cada PI; si un radio no cabe entre tangentes, no lo crea y lo informa. Devuelve la
lista de PI con radio, longitud de curva y progresiva.
```

2.3 · crear_perfil_terreno_y_vista SIN CAMBIOS

```
Crea el perfil de terreno de un alineamiento sobre una superficie y su vista de perfil.
Parámetros: alineamiento, superficie, estilos y punto de inserción (o automático, a la derecha
de lo dibujado).
```

2.4 · crear_rasante SIN CAMBIOS

```
Crea un perfil de diseño en una vista de perfil a partir de una lista de PVI (progresiva,
cota) y longitudes de curva vertical. Devuelve la pendiente de cada tramo y los datos de cada
curva. Con simular=true solo informa pendientes, sin crear nada.
```

2.5 · crear_corredor (con sus superficies) SIN CAMBIOS

```
Crea un corredor con un alineamiento, una rasante y un ensamblaje ya existente en el dibujo
(de mi plantilla), entre dos progresivas y con una frecuencia dada. Asigna la superficie de
terreno como objetivo de los taludes reutilizando la lógica de asignar_objetivos y reconstruye
el corredor. Luego crea las superficies de corredor «Top» y «Datum» (por códigos de vínculo,
con corrección de voladizos) para que calcular_volumenes tenga con qué comparar. Devuelve
estado_corredor y los nombres de las dos superficies. Si la sesión se hace muy larga, divídela
en dos ramas: corredor primero, superficies después.
```

2.6 · crear_secciones SIN CAMBIOS

```
Crea un grupo de líneas de muestreo cada [X] m con anchos izquierdo y derecho, y las vistas de
sección en cuadrícula desde un punto de inserción. Parámetros: alineamiento, intervalo, anchos,
estilo de vista y filas por columna.
```

2.7 · calcular_volumenes SIN CAMBIOS

```
Calcula los volúmenes de corte y relleno entre el terreno y la superficie «Datum» del
corredor. Totales: crea una superficie de volumen TIN (terreno vs. Datum) y devuelve corte,
relleno y neto, aplicando los factores de esponjamiento y compactación dados como parámetros.
Tabla por progresivas: si Civil 3D 2027 expone por API las listas de materiales y la tabla de
volúmenes, úsalas; si no, impleméntala con ejecutar_comando, documéntalo y avísame. Devuelve
también el nombre de la superficie de volumen creada.
```

Prompt 2.8 · Receta «flujo vial completo» AJUSTADO

DÓNDE: CLAUDE CODE WEB · Repos: civil3d-mcp + memoria-firma

```
Esta sesión tiene dos repositorios: civil3d-mcp (aquí trabajas, en la rama tarea/2.8-flujo) y
memoria-firma (solo para leer). Agrega a INSTRUCCIONES_AGENTE.md una receta «Flujo vial
completo» que encadene crear_superficie_tin, crear_alineamiento_desde_polilinea,
crear_perfil_terreno_y_vista, crear_rasante, crear_corredor (con sus superficies Top y Datum),
crear_secciones y calcular_volumenes. Primero todo con simular=true; después, paso a paso,
mostrando el resultado de cada paso y esperando mi confirmación antes del siguiente. Esta es la
única receta que va en INSTRUCCIONES_AGENTE.md, porque es el flujo principal.
Crea también herramientas-dev/VALIDACION_FLUJO_COMPLETO.md basada en la tarea 10 de
memoria-firma/evaluacion/tareas.yaml, escrita con datos genéricos (sin nombres ni coordenadas
reales, porque este repositorio es público). Abre el PR con "Para el agente instalador" y
"Para Jose".
```

Verificación de cada herramienta de la Etapa 2. Corre L.1 y luego el Prompt P con la rama: el agente sigue herramientas-dev/VALIDACION_[NOMBRE].md con el MCP sobre tu dibujo de prueba. Revisa el informe OK/FALLO. Mira en Civil 3D que el objeto se vea bien y que Ctrl+Z lo deshaga en un solo paso. Escribe «Verificación OK» en la sesión web, haz Merge, corre L.1 con main y repite 5.2-D y 5.2-PC.

MESES 4 A 6

9 Etapa 3 · ejecutar_codigo y recetas

Prompt 3.1 · ejecutar_codigo AJUSTADO

DÓNDE: CLAUDE CODE WEB · Repos: civil3d-mcp

```
Antes de programar, explícame los riesgos de esta herramienta y cómo los mitigas. Luego diseña e
implementa, en una rama, ejecutar_codigo(codigo_csharp, simular=true):
1. Compila el fragmento C# con Roslyn dentro del plugin, contra las DLL de Civil 3D 2027 y una
   clase de ayudas (Fachada) con lo básico: documento activo, transacción y búsqueda de objetos
   por nombre. Cada compilación va en un AssemblyLoadContext coleccionable.
2. Lista blanca: solo las DLL de AutoCAD/Civil 3D y lo básico de System. Nada de red
   (System.Net), procesos (System.Diagnostics.Process) ni archivos fuera de una carpeta
   permitida. Si el script lo intenta, no compila y el error lo dice. Pon esta revisión en
   ArbaMcp.Nucleo (Roslyn analiza el texto sin Civil 3D) con pruebas que corran aquí en la nube.
3. Se ejecuta siempre en el hilo principal de AutoCAD con el documento bloqueado.
4. Con simular=true: igual hace la copia de seguridad, ejecuta dentro de una transacción que se
   aborta al final y devuelve qué se habría creado o modificado, avisando que la simulación no
   es garantía total con objetos de Civil 3D y que la copia es la protección real.
5. Con simular=false aplica las reglas de Escritura.cs: copia, Deshacer, log y antes/después.
6. Tiempo límite cooperativo: un CancellationToken que la Fachada revisa en cada ayuda.
   Explícame en el informe por qué no se puede detener un script a la fuerza.
7. Errores de compilación con mensajes claros para el agente.
8. Desactivada por defecto; se activa con la variable de entorno ARBA_MCP_CODIGO=1.
En "Para el agente instalador": cómo poner ARBA_MCP_CODIGO=1 (variable de mi usuario en Windows,
y reiniciar Civil 3D) solo cuando yo lo pida, y cómo quitarlo.
```

Verificación: corre L.1 y el Prompt P. Con la herramienta desactivada, el MCP responde con un aviso. Pide al agente instalador que active ARBA_MCP_CODIGO=1: un script de prueba en simulación no deja nada en el dibujo y el informe dice qué habría creado; un script que use System.Net no compila.

Prompt 3.2 · Recetas AJUSTADO

DÓNDE: CLAUDE CODE WEB · Repos: civil3d-mcp + memoria-firma

```
Esta sesión tiene dos repositorios: civil3d-mcp (aquí programas, en una rama) y memoria-firma.
Implementa el sistema de recetas:
1. guardar_receta(nombre, pedido, codigo, parametros, notas): guarda en
   MEMORIA_FIRMA_DIR/recetas/ un .cs y un .yaml con pedido de ejemplo en español, parámetros,
   versión de Civil 3D, fecha y veces usada. Su respuesta me recuerda correr L.2 para subirla.
2. ejecutar_receta(nombre, parametros, simular): reutiliza el código guardado y suma 1 a veces
   usada.
3. buscar_api (también el comando) devuelve primero las recetas relevantes. Las recetas NO se
   copian a INSTRUCCIONES_AGENTE.md; ese archivo solo dice que existen y cómo buscarlas.
4. Una receta solo se guarda cuando yo lo apruebo explícitamente.
Pruebas aquí en la nube con una carpeta temporal de recetas.
```

Verificación: corre L.1. Con el Prompt P, pide al agente guardar una receta de prueba y corre L.2. En una conversación nueva del agente pide la misma tarea con otras palabras: el agente la encuentra y la reutiliza.

Prompt 3.3 · Promoción de recetas (cada mes) AJUSTADO

DÓNDE: CLAUDE CODE WEB · Repos: memoria-firma + civil3d-mcp

Antes de esta sesión corre L.2 en tu PC para que los contadores de uso estén al día.

```
Esta sesión tiene dos repositorios: memoria-firma y civil3d-mcp (solo para leer). No modifiques
código. Revisa memoria-firma/recetas/ y lista las recetas usadas 3 veces o más. Propón cuáles
conviene convertir en herramientas formales del plugin, con su especificación lista para la
plantilla de la Etapa 2.
```

MESES 6 A 8

10 Etapa 4 · Motor normativo DG-2018

Antes de empezar: copia el PDF oficial de la DG-2018 (vigente, con sus modificatorias) en C:\Proyectos\memoria-firma\normas\fuentes\ y corre L.2, o súbelo directo en GitHub (Add file → Upload files). Calcula a mano 5 casos de prueba (por ejemplo, radio mínimo para 3 velocidades).

Prompt 4.1 · Tablas y fórmulas de la norma AJUSTADO

DÓNDE: CLAUDE CODE WEB · Repos: memoria-firma

```
Esta sesión es sobre memoria-firma. En normas/fuentes/ está el PDF oficial de la DG-2018.
Instala con pip lo que necesites para leerlo (pdfplumber, PyMuPDF). Si una tabla del PDF es una
imagen escaneada, dímelo y no adivines valores.
Extrae a normas/tablas/dg2018.sqlite las tablas de diseño geométrico que necesito: velocidad de
diseño, radios mínimos y peraltes, longitudes de transición, distancias de visibilidad de parada
y adelantamiento, pendientes máximas y longitudes de curvas verticales.
Cada valor guarda su referencia: número de tabla, sección y página. Guarda también las fórmulas
de la norma (por ejemplo, radio mínimo en función de velocidad, peralte máximo y fricción
máxima; longitud de curva vertical con el índice K) con su sección, no solo los valores.
Anota en normas/tablas/VERSION.md la versión del documento y las modificatorias incluidas.
Guarda el script de extracción en normas/tablas/ para poder repetirlo.
Después crea normas/tablas/VERIFICAR.md con: (a) TODAS las filas de las tablas de radios
mínimos y peraltes, visibilidad, pendientes máximas y curvas verticales, en el mismo orden que
el PDF, y junto a cada tabla la imagen recortada de su página (normas/tablas/verificar_img/),
para que yo compare lado a lado en GitHub; y (b) 20 valores al azar del resto. No uses ningún
valor que no esté en el PDF. Haz commit y push.
```

Verificación: abre normas/tablas/VERIFICAR.md en GitHub y compara las tablas críticas completas con la imagen de cada página. Si una columna está corrida o un valor falla, pide en la misma sesión que vuelva a extraer toda esa tabla. No sigas a 4.2 con menos del 100 % correcto en las tablas críticas.

Prompt 4.2 · Herramienta verificar_norma AJUSTADO

DÓNDE: CLAUDE CODE WEB · Repos: civil3d-mcp + memoria-firma

```
Esta sesión tiene dos repositorios: civil3d-mcp (aquí programas, en una rama) y memoria-firma.
Agrega al puente MCP la herramienta verificar_norma(norma, tipo_elemento, nombre). Para un
alineamiento o una rasante del dibujo (leídos con las herramientas existentes), compara cada
curva y tramo con MEMORIA_FIRMA_DIR/normas/tablas/dg2018.sqlite y devuelve una tabla: elemento,
valor de diseño, valor exigido, Cumple / No cumple y referencia de la norma.
Las fórmulas van en código, con pruebas pytest basadas en estos casos que calculé a mano:
[pega tus 5 casos de prueba]
Las pruebas de civil3d-mcp usan una tabla pequeña de ejemplo; como memoria-firma está en esta
sesión, corre además mis 5 casos contra la base real y muéstrame el resultado.
Si el valor de la tabla y el de la fórmula difieren, informa ambos y usa el de la tabla.
Si falta un dato (velocidad de diseño, clasificación de la vía, orografía), pregúntalo; nunca
lo supongas.
```

Verificación: corre L.1 y el Prompt P. Sobre un eje de prueba con una curva de radio menor al mínimo a propósito, la herramienta marca esa curva como No cumple y cita la tabla.

Prompt 4.3 · Norma dentro del alineamiento AJUSTADO

DÓNDE: CLAUDE CODE WEB · Repos: civil3d-mcp + memoria-firma

```
Conecta verificar_norma con crear_alineamiento_desde_polilinea, en una rama: con un parámetro
velocidad_diseno, la herramienta propone el radio mínimo de la norma en cada PI y, al terminar,
ejecuta verificar_norma y adjunta el resultado. Agrega su caso a VALIDACION y abre el PR con
"Para el agente instalador" y "Para Jose".
```

Verificación: corre L.1 y el Prompt P. Pide crear un alineamiento con velocidad_diseno: propone el radio mínimo en cada PI y adjunta la verificación.

11 Fichas de experiencia (continuo)

Prompt F · Nueva ficha desde la nube REUTILIZABLE

DÓNDE: CLAUDE CODE WEB · Repos: memoria-firma

Quita nombres de proyecto y de cliente y coordenadas reales antes de pegar la nota.

```
Esta sesión es sobre memoria-firma. Escribe una ficha nueva en casos/ con PLANTILLA_FICHA.md a
partir de esto (ya quité nombres y coordenadas):
[pega la nota de voz transcrita o las líneas del cuaderno de obra]
Corre validar_fichas.py. Si falla, dime por qué y corrige solo lo que yo apruebe. Si pasa, haz
commit en main y push.
```

Verificación: la ficha aparece en memoria-firma/casos/ y la Action de GitHub queda en verde.

12 Reglas fijas (v1.3)

- Un prompt por sesión, siempre con el plan aprobado antes (modo Plan).
- Nada se fusiona a main (Merge) sin que la verificación en Civil 3D haya pasado.
- Dibujos de prueba solo con datos propios. Nada real en el repositorio público, tampoco en commits, nombres de rama ni descripciones de PR.
- Si Claude Code propone cambiar archivos que no tienen que ver con la tarea, detenlo (regla 9 de CLAUDE.md).
- Toda simulación va acompañada de copia de seguridad: abortar una transacción no es garantía con objetos de Civil 3D.
- Nada se promueve sin ganar en el set de evaluación: ni una receta a herramienta, ni un buscador propio a producción.
- **NUEVA:** el código solo se arregla en Claude Code web. El agente instalador instala, corre y reporta.
- **NUEVA:** en las pruebas el agente usa solo el MCP civil3d. Si habla con Civil 3D por curl, scripts o el puerto 8765, la prueba no vale.
- **NUEVA:** lo que se guarda en tu PC (fichas, recetas, informes, dataset, catálogo) llega a GitHub solo cuando corres L.2.

13 Ajustes a la hoja de ruta general «IA especialista en ingeniería» v1.1

Parte	Antes (v1.1)	Ahora (trabajo en GitHub)
Fase 0 · Preparar la PC	Claude Desktop o Claude Code, Visual Studio Community, Python y repositorio privado	Claude Code web para el código; el agente instalador instala, corre y prueba con el MCP civil3d. Basta el .NET 10 SDK (confirmado en L.0), no hace falta Visual Studio.
Sección 4 · Día a día, paso 4 «Ejecución»	En transacción, sobre copia	Igual, pero siempre en tu PC, por el MCP civil3d de tu agente. La nube nunca toca Civil 3D.
Memoria de la firma	Repositorio privado	memoria-firma en GitHub (privado, con la Claude GitHub App instalada). Se sube con L.2 y Claude Code web la lee cuando la agregas a la sesión.
Fase 4 · Buscador propio	Entrenar con más de 1000 pares	Claude Code web escribe el código de entrenamiento. El entrenamiento corre en tu PC con tarjeta RTX, guiado por el agente instalador: la máquina de la nube no tiene tarjeta gráfica y es chica (4 CPU, 16 GB).

Parte	Antes (v1.1)	Ahora (trabajo en GitHub)
Fase 5 · SAP2000 y Revit	Segundo software con su catálogo	Mismo ciclo de este documento: código en la nube, instalación y pruebas en tu PC.
Reglas fijas	6 reglas	Se suman las 3 reglas nuevas de la sección 12.